

Tree Species

Table of contents

1	Exploratory Data Analysis	1
1.1	Summary	9
2	Clustering	11
2.1	Encoding	11
2.2	Scaling	11
2.3	k-means clustering	11
2.3.1	Description of clusters	14
2.3.2	Principal components	16
2.3.3	Scatter plot summary	17
2.4	Visualisation of cluster allocation using a Decision Tree	17
2.5	Decision Tree analysis	18

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from adjustText import adjust_text
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
# Set plot style
sns.set_theme(style="whitegrid")
```

1 Exploratory Data Analysis

```
# Load tree species dataset
tree_df = pd.read_csv("../data/species.csv")
```

```
# Display the first few rows
print("First 5 rows:")
print(tree_df.head())
```

First 5 rows:

	id	name	common_name	rainfall_mm_min	rainfall_mm_max	\
0	1	Casuarina equisetifolia	Ai-kakeu	200	3500	
1	2	Tectona grandis	Ai-teka	1500	2000	
2	3	Pterocarpus indicus	Ai-na	900	2200	
3	4	Santalum album	Ai-kamelli	625	1625	
4	5	Sterculia foetida	Ai-nitas	1100	1800	

	temperature_celsius_min	temperature_celsius_max	elevation_m_min	\
0	10	35	0	
1	16	40	0	
2	22	32	0	
3	10	35	50	
4	18	32	0	

	elevation_m_max	ph_min	ph_max	soil_textures	\
0	1400	5.0	7.7	sand, loamy sand, sandy loam, loam	
1	1200	6.5	7.5	loam	
2	600	6.0	7.0	sand, clay loam	
3	1200	6.0	7.5	sand, loamy sand, sandy loam, loam	
4	1000	6.0	7.5	sand, loamy sand, sandy loam, loam	

	coastal	riparian	nitrogen_fixing	shade_tolerant	bank_stabilising	\
0	True	False	True	False	True	
1	False	False	False	False	True	
2	False	True	True	False	True	
3	False	False	False	False	False	
4	False	False	True	False	True	

	agroforestry_types
0	intercropping, block, boundary
1	block, boundary
2	intercropping, boundary
3	intercropping, boundary
4	boundary

```
# Display basic information about the dataset
print("\nDataset Info:")
print(tree_df.info())
```

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 20 entries, 0 to 19
```

Data columns (total 18 columns):

#	Column	Non-Null Count	Dtype
0	id	20 non-null	int64
1	name	20 non-null	object
2	common_name	20 non-null	object
3	rainfall_mm_min	20 non-null	int64
4	rainfall_mm_max	20 non-null	int64
5	temperature_celsius_min	20 non-null	int64
6	temperature_celsius_max	20 non-null	int64
7	elevation_m_min	20 non-null	int64
8	elevation_m_max	20 non-null	int64
9	ph_min	20 non-null	float64
10	ph_max	20 non-null	float64
11	soil_textures	20 non-null	object
12	coastal	20 non-null	bool
13	riparian	20 non-null	bool
14	nitrogen_fixing	20 non-null	bool
15	shade_tolerant	20 non-null	bool
16	bank_stabilising	20 non-null	bool
17	agroforestry_types	20 non-null	object

dtypes: bool(5), float64(2), int64(7), object(4)

memory usage: 2.3+ KB

None

```
# Select all numeric types (floats and integers)
```

```
tree_numerical_cols =
```

```
↳ tree_df.select_dtypes(include=[np.number]).columns.tolist()
```

```
# Remove the id column
```

```
tree_numerical_cols.remove("id")
```

```
# Select object (string) types
```

```
tree_categorical_cols =
```

```
↳ tree_df.select_dtypes(include=["object"]).columns.tolist()
```

```
# Select boolean types
```

```
tree_boolean_cols = tree_df.select_dtypes(include=["bool"]).columns.tolist()
```

```
# Filter for columns with low cardinality (few unique values)
```

```
# This prevents plotting unique identifiers like 'species_name'
```

```
tree_categorical_cols = [col for col in tree_categorical_cols if
```

```
↳ tree_df[col].nunique() < 15]
```

```
# Display descriptive statistics
```

```
print("\nDescriptive Statistics:")
```

```
print(tree_df.describe())
```

Descriptive Statistics:

	id	rainfall_mm_min	rainfall_mm_max	temperature_celsius_min	\
count	20.00000	20.000000	20.000000	20.000000	
mean	10.50000	1075.750000	2906.250000	19.200000	
std	5.91608	563.025836	1092.675224	4.926833	
min	1.00000	200.000000	1200.000000	10.000000	
25%	5.75000	722.500000	2000.000000	16.750000	
50%	10.50000	1000.000000	2750.000000	20.000000	
75%	15.25000	1500.000000	4000.000000	22.000000	
max	20.00000	2500.000000	5000.000000	28.000000	

	temperature_celsius_max	elevation_m_min	elevation_m_max	ph_min	\
count	20.000000	20.00000	20.000000	20.000000	
mean	32.500000	72.50000	1247.500000	5.395000	
std	3.456497	171.27617	681.228573	0.754966	
min	28.000000	0.00000	150.000000	4.000000	
25%	29.500000	0.00000	950.000000	5.000000	
50%	32.000000	0.00000	1200.000000	5.500000	
75%	35.000000	0.00000	1500.000000	6.000000	
max	40.000000	500.00000	3000.000000	6.500000	

	ph_max
count	20.000000
mean	7.475000
std	0.437547
min	6.500000
25%	7.300000
50%	7.500000
75%	7.550000
max	8.500000

```
# Check for missing values
print("\nMissing Values:")
print(tree_df.isnull().sum())
```

Missing Values:

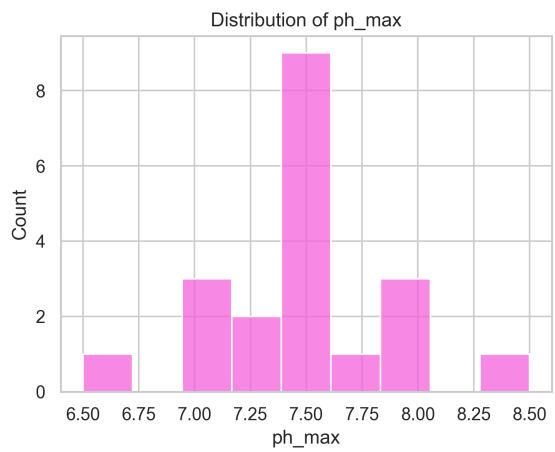
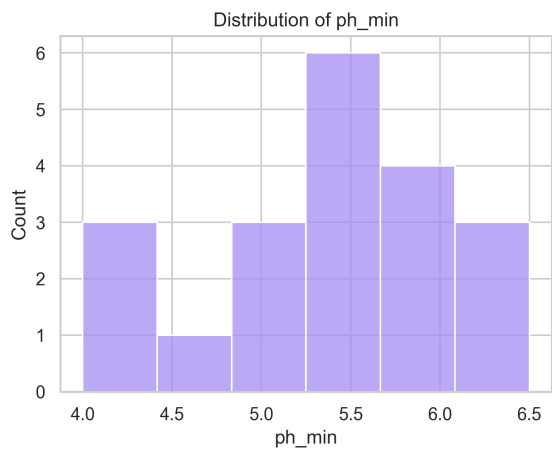
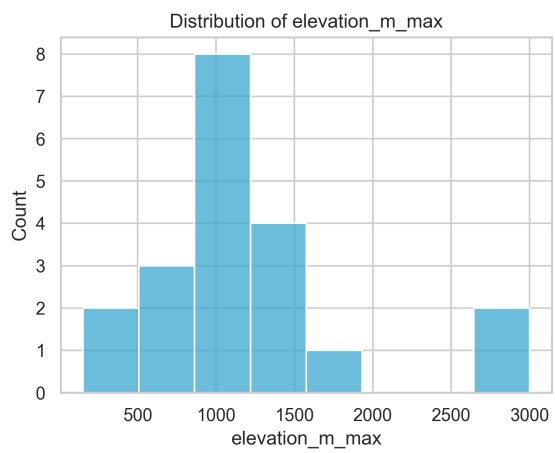
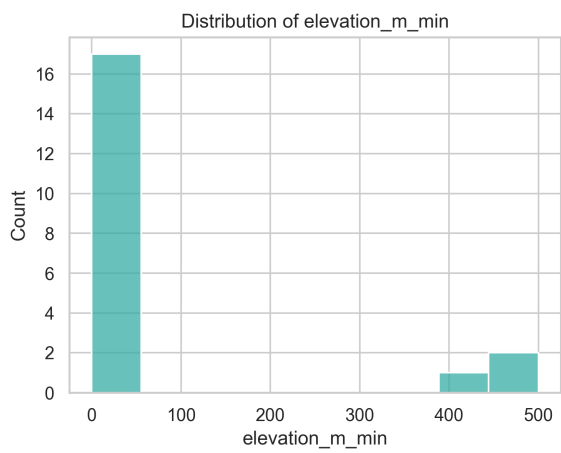
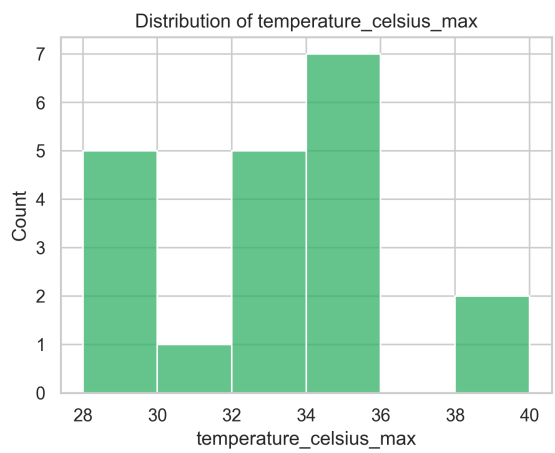
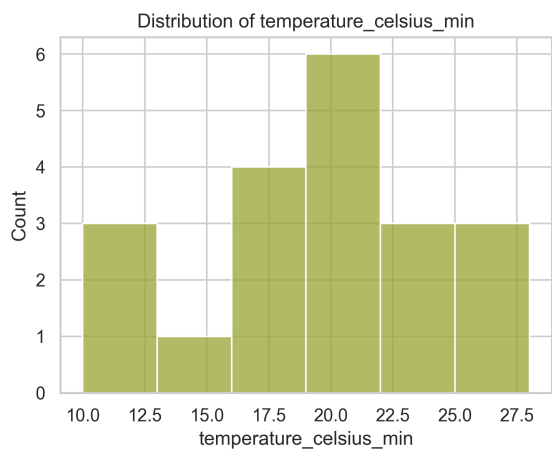
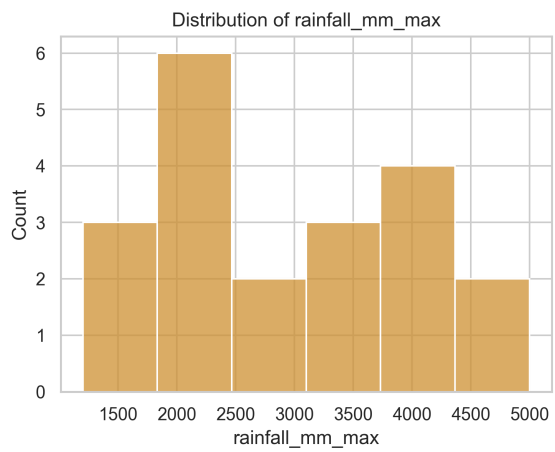
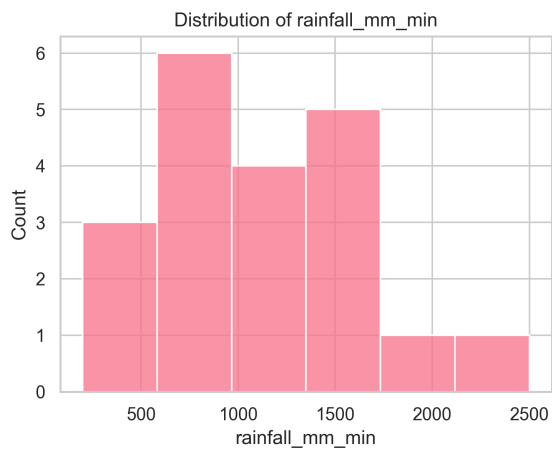
id	0
name	0
common_name	0
rainfall_mm_min	0
rainfall_mm_max	0
temperature_celsius_min	0
temperature_celsius_max	0
elevation_m_min	0
elevation_m_max	0
ph_min	0
ph_max	0
soil_textures	0

```
coastal          0
riparian         0
nitrogen_fixing  0
shade_tolerant   0
bank_stabilising 0
agroforestry_types 0
dtype: int64
```

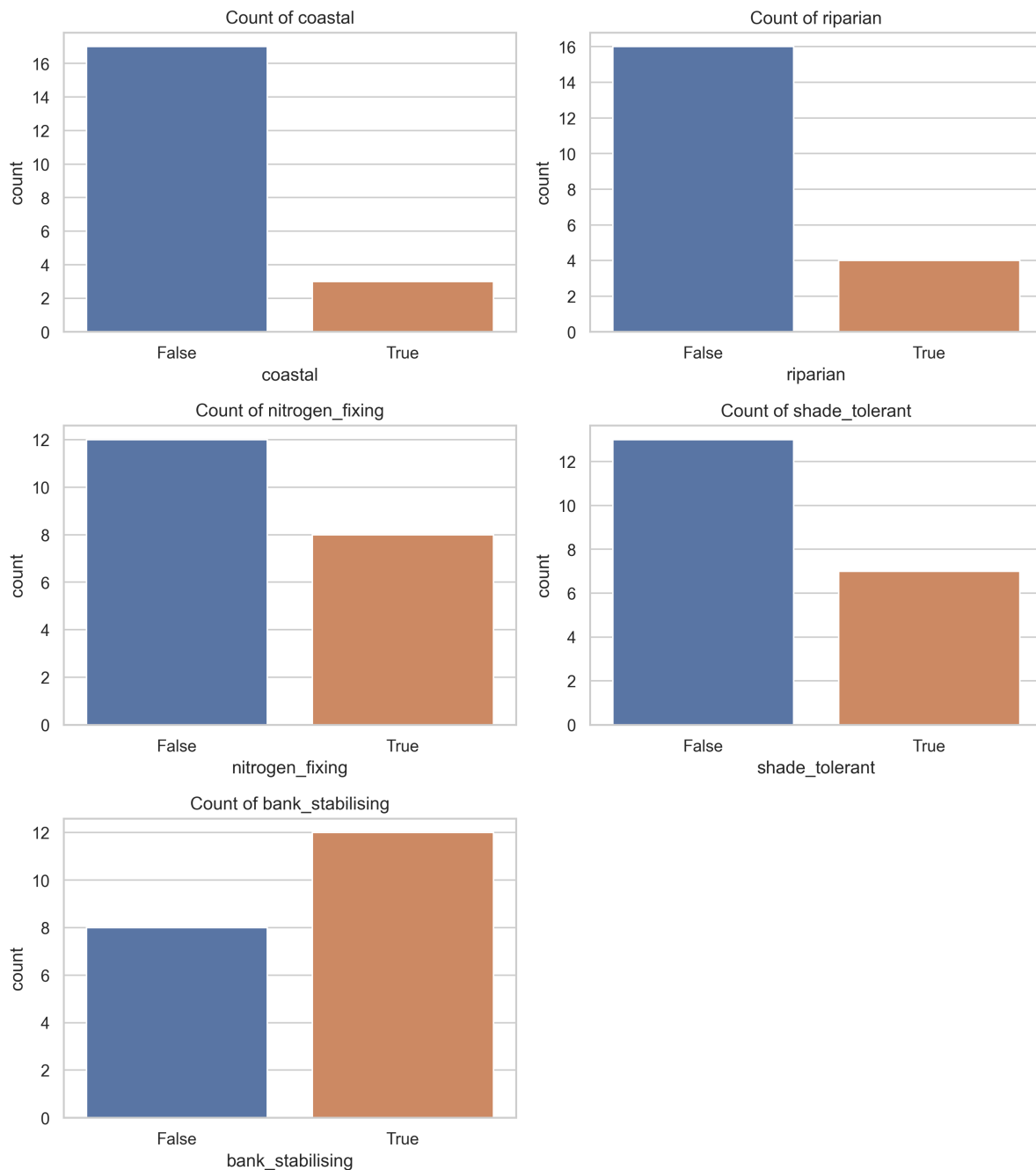
```
# Create histograms for numerical columns

# Generate a palette with enough unique colours
tree_colours = sns.color_palette("husl", len(tree_numerical_cols))

plt.figure(figsize=(10, 16))
for i, col in enumerate(tree_numerical_cols):
    plt.subplot(4, 2, i + 1)
    sns.histplot(tree_df[col], kde=False, color=tree_colours[i])
    plt.title(f"Distribution of {col}")
    plt.tight_layout()
```



```
# Create bar charts for boolean columns
plt.figure(figsize=(10, 18))
for i, col in enumerate(tree_boolean_cols, 1):
    plt.subplot(5, 2, i)
    sns.countplot(x=tree_df[col], palette="deep", hue=tree_df[col],
    ↪ legend=False)
    plt.title(f"Count of {col}")
    plt.tight_layout()
```



```
def process_and_plot_list_column(df, column_name, title, colour_palette):
    """
```

Function to process list-like columns and plot a horizontal bar plot.

```
:param df: Dataframe containing the data
:param column_name: Name of the column to plot
:param title: String for plot title
:param colour_palette: Seaborn colour palette
:returns: None
"""

# For the specified column, split the string by a comma to get a list
# Explode the list, so each item gets its own row
# Strip whitespace from the strings
series = df[column_name].str.split(",").explode().str.strip()

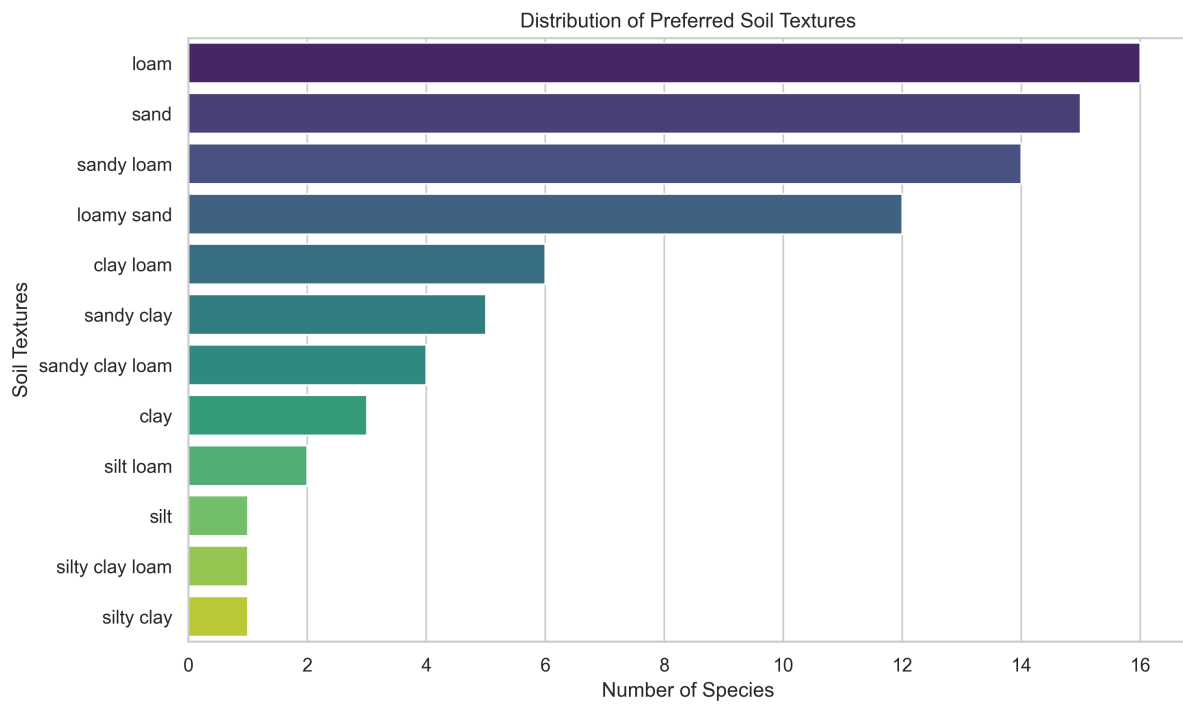
# Count the occurrences
counts = series.value_counts()

# Create DataFrame for plotting
plot_df = counts.reset_index()
plot_df.columns = ["Category", "Count"]

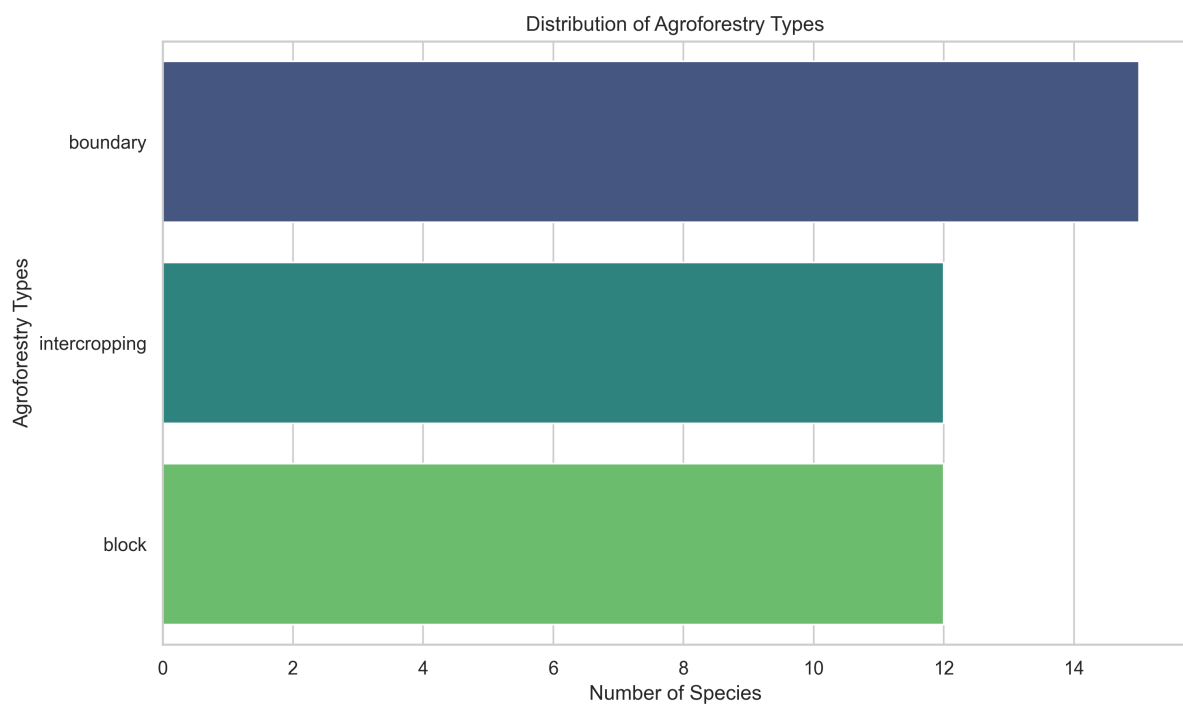
# Plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=plot_df,
    x="Count",
    y="Category",
    palette=colour_palette,
    hue="Category",
    legend=False,
)
plt.title(title)
plt.xlabel("Number of Species")
plt.ylabel(column_name.replace("_", " ").title())
plt.tight_layout()

return None
```

```
# Process 'preferred_soil_texture'
process_and_plot_list_column(
    tree_df,
    "soil_textures",
    "Distribution of Preferred Soil Textures",
    "viridis",
)
```



```
# Process 'agroforestry_types'
process_and_plot_list_column(tree_df, "agroforestry_types", "Distribution of
↪ Agroforestry Types", "viridis")
```



1.1 Summary

Data Structure:

- The dataset contains 20 entries and 18 columns.
- Columns:
 - Identifiers: `species_id`, `species_name`, `species_common_name`
 - Environmental Factors (Numerical): `rainfall_mm_min/max`, `temperature_celsius_min/max`, `elevation_m_min/max`, `ph_min/max`
 - Characteristics (Categorical/Boolean): `preferred_soil_texture`, `costal`, `riparian`, `nitrogen_fixing`, `shade_tolerant`, `bank_stabilising`, `agroforestry_types`

Data Quality:

- There are no missing values in any of the columns.
- Data types appear appropriate (integers/floats for ranges, booleans for flags).

Distributions & Statistics:

- Rainfall: Minimum rainfall requirements range from 200 mm to 2500 mm, while maximums go up to 5000 mm.
- Temperature: Minimum temperatures range from 10 °C to 28 °C, and maximums from 28 °C to 40 °C.
- Elevation: Most species start at 0 m elevation, with some tolerating up to 3000 m. There are a few species that require at least 400 m.
- pH: Soil pH preferences generally span from acidic (min 4.0) to slightly alkaline (max 8.5).

Visualisations:

- Numerical Distributions: The histograms show the spread of environmental tolerances. For example, `elevation_m_min` is heavily skewed towards 0, indicating many species can grow at sea level. There are only two species that can grow at high elevation.
- Boolean Counts: The bar charts show the prevalence of traits like nitrogen fixing, shade tolerance and bank stabilising. They also highlight that there are not many species suitable for coastal or riparian environments.
- Categorical Counts: The horizontal bar charts for the preferred soil texture and agroforestry types show:
 - Preferred Soil Textures:
 - * Loam, Sand, and Sandy Loam are the most common preferred soil types, suitable for the majority of species in this dataset.
 - * Finer textures like Silt and Silty Clay are less commonly supported.
 - Agroforestry Types:
 - * Boundary planting is the most versatile use, applicable to 15 species.
 - * Intercropping and Block planting are also widely supported, each suitable for 12 species.

2 Clustering

2.1 Encoding

```
# Encode the boolean feature as 0 and 1
for col in tree_boolean_cols:
    tree_df[col] = tree_df[col].astype(str).str.strip().map({"True": 1,
↪  "False": 0, "true": 1, "false": 0}).fillna(0).astype(int)

# List-like columns (One-hot encoding)
def get_dummies_from_list_col(df, col):
    normalised = df[col].str.replace(" ", ",")
    dummies = normalised.str.get_dummies(sep=",")
    dummies = dummies.add_prefix(f"{col}_")
    return dummies

tree_soil_dummies = get_dummies_from_list_col(tree_df, "soil_textures")
tree_agro_dummies = get_dummies_from_list_col(tree_df, "agroforestry_types")

# Combine all features
tree_features_df = pd.concat(
    [
        tree_df[tree_numerical_cols],
        tree_df[tree_boolean_cols],
        tree_soil_dummies,
        tree_agro_dummies,
    ],
    axis=1,
)
```

2.2 Scaling

```
scaler = StandardScaler()
tree_scaled_features = scaler.fit_transform(tree_features_df)
```

2.3 k-means clustering

```
# Determine optimal k (Silhouette Score)
tree_best_k = 2
tree_best_score = -1

for k in range(2, 6):
```

```

tree_kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = tree_kmeans.fit_predict(tree_scaled_features)
score = silhouette_score(tree_scaled_features, labels)
if score > tree_best_score:
    tree_best_score = score
    tree_best_k = k

print(f"Optimal number of clusters (k) found: {tree_best_k} (Silhouette Score:
↪ {tree_best_score:.3f})")

```

Optimal number of clusters (k) found: 2 (Silhouette Score: 0.187)

```

# Apply K-Means using best k
tree_kmeans = KMeans(n_clusters=tree_best_k, random_state=42, n_init=10)
tree_df["Cluster"] = tree_kmeans.fit_predict(tree_scaled_features)

```

```

# Get the centroids of each cluster
tree_centroids = pd.DataFrame(tree_kmeans.cluster_centers_,
↪ columns=tree_features_df.columns)

# Calculate the difference between Clusters
# Get the absolute value difference
tree_diff = tree_centroids.diff().iloc[1].abs()

# Sort to find the most important features
tree_sorted_diff = tree_diff.sort_values(ascending=False)

# Display the results
print("Top features driving the species cluster separation:")
print(tree_sorted_diff.head(14))

```

Top features driving the species cluster separation:

soil_textures_clay loam	1.909407
soil_textures_sandy clay loam	1.718750
soil_textures_silt loam	1.666667
rainfall_mm_min	1.478876
agroforestry_types_boundary	1.443376
soil_textures_sand	1.443376
soil_textures_silty clay	1.147079
soil_textures_silt	1.147079
soil_textures_silty clay loam	1.147079
shade_tolerant	1.048285
rainfall_mm_max	0.990309
riparian	0.937500
bank_stabilising	0.893043
soil_textures_loamy sand	0.893043

Name: 1, dtype: float64

Soil texture dominates the list of features driving the separation between the clusters. However, because the soil texture often correlates to how the trees are used and the environment they are planted in, I will focus on the environmental and usage features from the importance lists to describe the clustering.

```
# Feature for investigation
tree_features_of_interest = [
    "rainfall_mm_min",
    "bank_stabilising",
    "riparian",
    "shade_tolerant",
]

# Add 'boundary' for analysis
tree_df["agroforestry_boundary"] =
    ↪ tree_agro_dummies["agroforestry_types_boundary"]
tree_features_of_interest.append("agroforestry_boundary")

# Calculate averages per cluster
tree_summary = tree_df.groupby("Cluster")[tree_features_of_interest].mean().T
tree_summary.columns = ["Cluster 0", "Cluster 1"]

print("Key Statistics for Species Classification:")
print(tree_summary)
```

Key Statistics for Species Classification:

	Cluster 0	Cluster 1
rainfall_mm_min	913.4375	1725.00
bank_stabilising	0.6875	0.25
riparian	0.1250	0.50
shade_tolerant	0.2500	0.75
agroforestry_boundary	0.8750	0.25

Feature	Cluster 0	Cluster 1	Interpretation
Min Rainfall	913 mm	1725 mm	Cluster 0 species can survive with much less water, so are hardy. Cluster 1 requires high rainfall. The majority of Cluster 0 species can be used for stabilising banks. Cluster 1 has a much higher concentration of river-dwelling (Riparian) species.
Bank Stabilising	69%	25%	
Riparian	12%	50%	

Feature	Cluster 0	Cluster 1	Interpretation
Shade Tolerant	25%	75%	Most Cluster 1 species can grow in the shade (Shade Tolerant), unlike Cluster 0.
Boundary Planting	88%	25%	Almost all Cluster 0 species are suitable for boundary planting.

2.3.1 Description of clusters

- Cluster 0 - Hardy, Boundary and Stabilising
- Cluster 1 - High Rainfall, Riparian & Shade Tolerant

```
# PCA for Visualisation
tree_pca = PCA(n_components=2)
tree_principal_components = tree_pca.fit_transform(tree_scaled_features)
tree_pca_df = pd.DataFrame(data=tree_principal_components, columns=["PC1",
↪ "PC2"])
tree_pca_df["Cluster"] = tree_df["Cluster"]
tree_pca_df["Species"] = tree_df["name"]
```

```
# Get the correlation between original features and the PCs
tree_loadings = pd.DataFrame(tree_pca.components_.T, columns=["PC1", "PC2"],
↪ index=tree_features_df.columns)

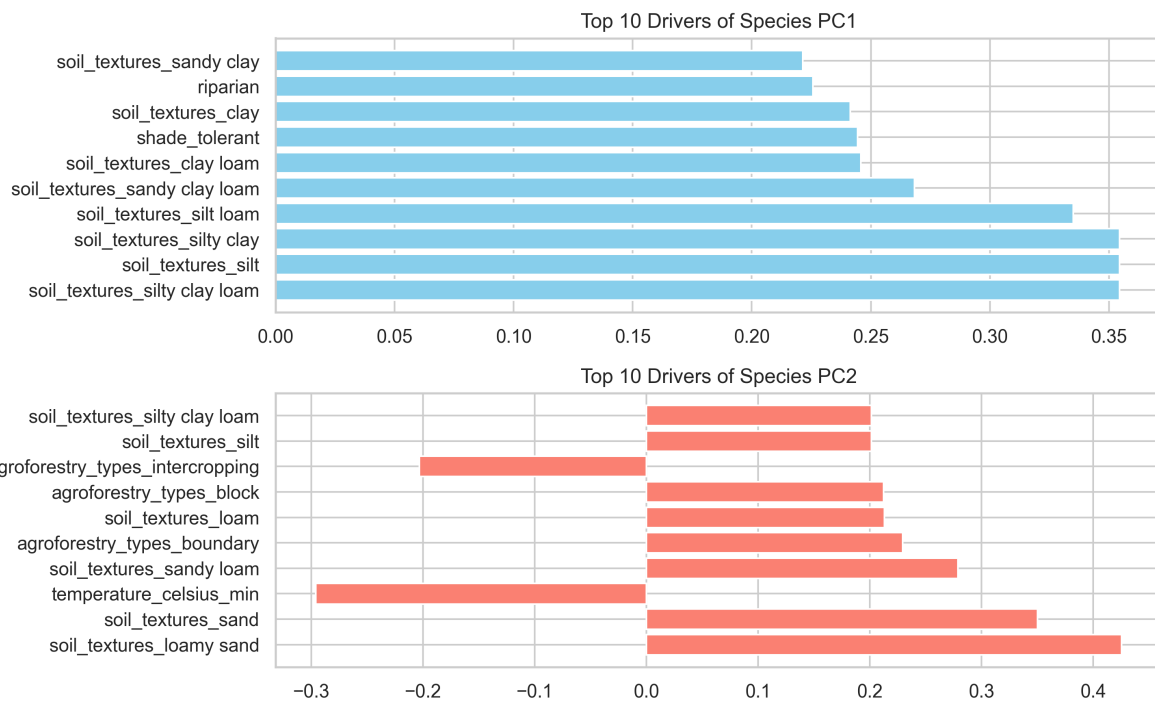
# Sort to find the top drivers for each component
tree_top_pc1 =
↪ tree_loadings.reindex(tree_loadings["PC1"].abs().sort_values(ascending=False).index).head(
tree_top_pc2 =
↪ tree_loadings.reindex(tree_loadings["PC2"].abs().sort_values(ascending=False).index).head(
```

```
# Plot
plt.figure(figsize=(10, 6))

plt.subplot(2, 1, 1)
plt.barh(tree_top_pc1.index, tree_top_pc1["PC1"], color="skyblue")
plt.title("Top 10 Drivers of Species PC1")

plt.subplot(2, 1, 2)
plt.barh(tree_top_pc2.index, tree_top_pc2["PC2"], color="salmon")
plt.title("Top 10 Drivers of Species PC2")

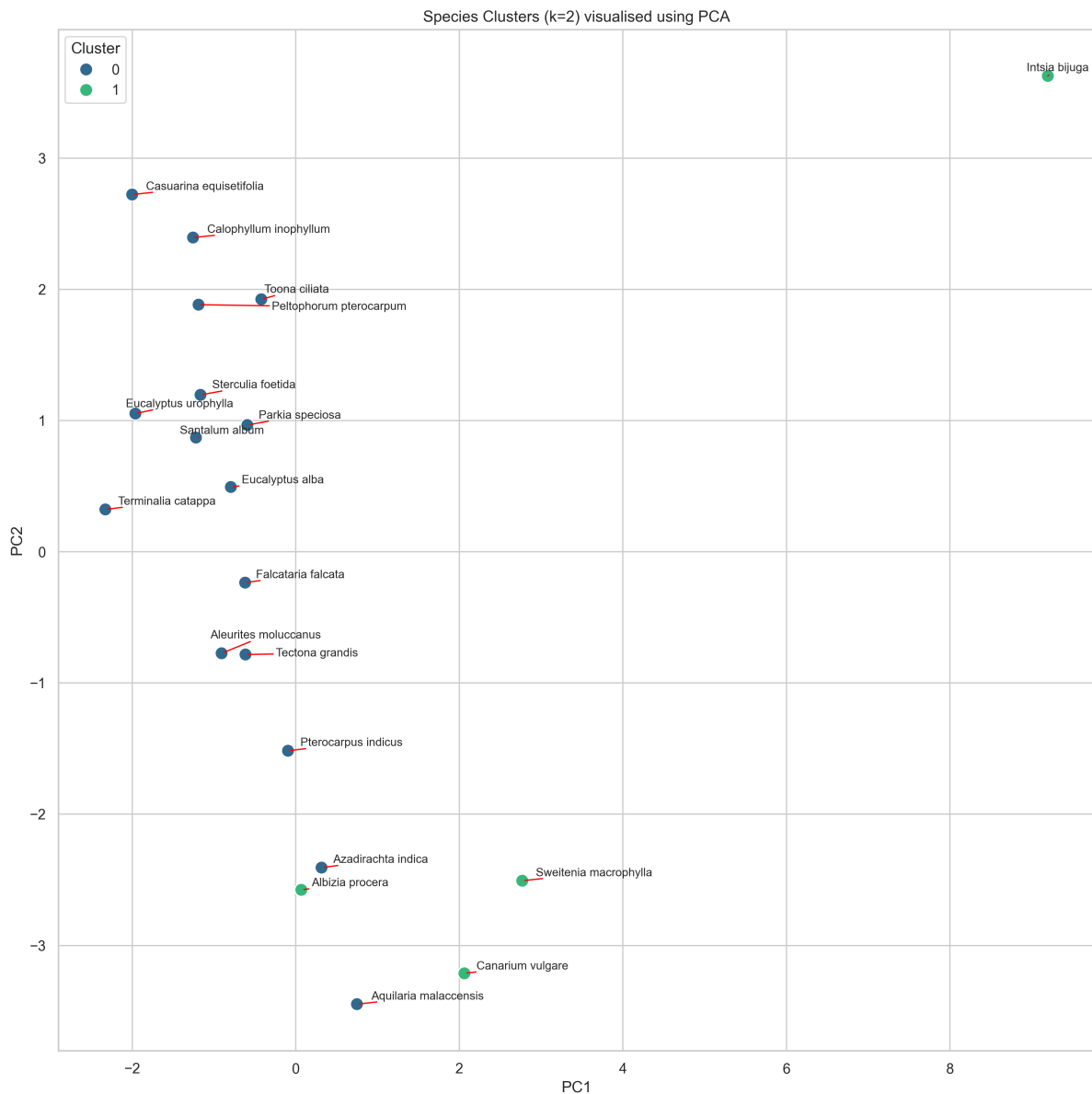
plt.tight_layout()
```



```
# Scatter plot of trees using PC1 and PC2 as axes
plt.figure(figsize=(12, 12))
sns.scatterplot(x="PC1", y="PC2", hue="Cluster", data=tree_pca_df,
               ↪ palette="viridis", s=100)
plt.title(f"Species Clusters (k={tree_best_k}) visualised using PCA")

texts = []
# Annotate points
for i in range(tree_pca_df.shape[0]):
    texts.append(plt.text(tree_pca_df.PC1[i], tree_pca_df.PC2[i],
    ↪ tree_pca_df.Species[i], fontsize=9))

adjust_text(texts, arrowprops=dict(arrowstyle="-", color="red"))
plt.tight_layout()
plt.show()
```



2.3.2 Principal components

PC1 seems to capture the difference between species that need fine, silty soils vs those that are hardy stabilisers.

- Positive Side (+): Strongly driven by Silt and Clay textures (**silty clay loam, silt, silt loam**). If a species is far to the right, it likely needs these specific, finer soils.
- Negative Side (-): Driven by Bank Stabilising and Coastal traits. If a species is far to the left, it is likely a hardy coastal or riverbank stabiliser.

PC2 separates species based on their preference for sandy soils vs those that need high temperatures/rainfall.

- Positive Side (+): Dominated by Sand and Loamy Sand. Species high up on the chart are sand-lovers.

- Negative Side (-): Driven by Temperature (Min), Rainfall, and Shade Tolerance. Species low down on the chart tend to be heat-loving, shade-tolerant, or high-rainfall species.

2.3.3 Scatter plot summary

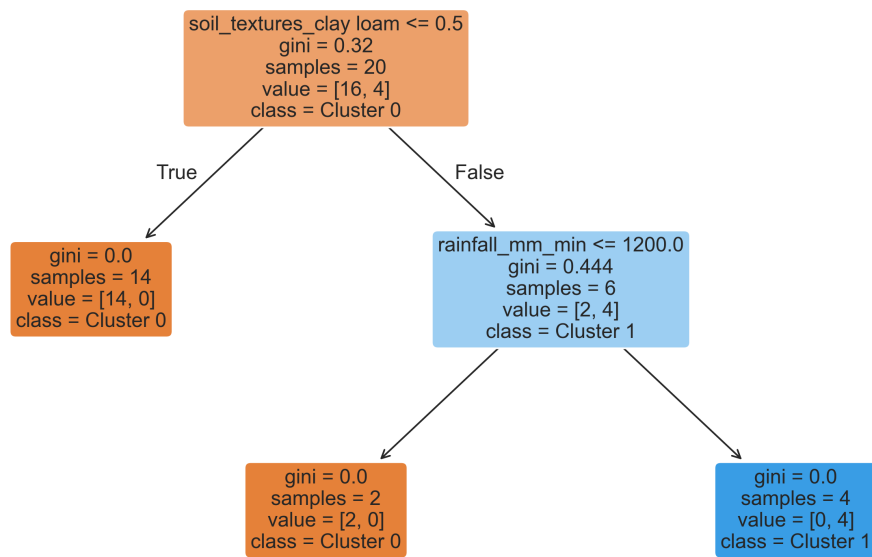
- Moving Right ($PC1 > 0$): Species that prefer complex, silty/clay soils.
- Moving Left ($PC1 < 0$): Tough, bank-stabilising species.
- Moving Up ($PC2 > 0$): Find the sand-loving species.
- Moving Down ($PC2 < 0$): Find the heat/rain-loving, shade-tolerant species.

2.4 Visualisation of cluster allocation using a Decision Tree

```
# Train Decision Tree
# X = Original Features (for readable rules), y = Cluster Labels
tree_clf = DecisionTreeClassifier(max_depth=3, random_state=1)
tree_clf.fit(tree_features_df, tree_df["Cluster"])

# Plot the Tree
plt.figure(figsize=(10, 6))
plot_tree(
    tree_clf,
    feature_names=tree_features_df.columns,
    class_names=["Cluster 0", "Cluster 1"],
    filled=True,
    rounded=True,
    fontsize=12,
)
plt.title("Decision Tree explaining species cluster allocations")
plt.tight_layout()
plt.show()
```

Decision Tree explaining species cluster allocations



2.5 Decision Tree analysis

The decision tree confirms that the separation between the two species groups is driven by just two key variables: Soil Texture and Rainfall.

- Does the species tolerate Clay Loam?
 - No: The species is immediately assigned to Cluster 0 (Hardy/Boundary). This suggests that Cluster 0 species are generally defined by not requiring this specific heavy soil type (they tend to prefer sand/loam).
 - Yes: If the species does like Clay Loam, move to the next check.
- Is the Minimum Rainfall > 1200 mm?
 - No: Even if it likes Clay Loam, if it can survive on less than 1200 mm of rain, it is assigned to Cluster 0. This reinforces Cluster 0 as the “Hardy” group.
 - Yes: If it likes Clay Loam and requires high rainfall (>1200 mm), it is assigned to Cluster 1 (High Rainfall/Riparian).